

```

A2A multiagent execution examples
# This script has multiple examples.
# You can execute this script as such and in a terminal (no need to restart
for each example, just once is enough)
# You can enter the curl commands given under each example to see what A2A
platform can do.
# There are two ways to monitor the status of the tasks. First, use the
corresponding curl commands given under each example or use the same curl
command template with correct task_id and agent_id. Second, can be watched
on a browser endpoint of the form agents/<agent_id>/tasks or can use the
corresponding browser endpoints given under each example.
# task_id can be found in the response message given by the server after
submitting the jobs.
# Parts of code other than examples are generally used for every agent or
platform.

# import sys
# sys.path.append("your/path/to/BESSER-Agentic-Framework") # If you clone
this repository, then add the location to BESSER-Agentic-Framework here.

import asyncio
from aiohttp import web

from baf.core.agent import Agent
from baf.exceptions.logger import logger
from baf.platforms.a2a.agent_registry import AgentRegistry
from baf.platforms.a2a.server import create_app

# Create a registry of Agents
registry = AgentRegistry()

# Define each agent
agent1 = Agent('TestAgent1')
agent2 = Agent('TestAgent2')
agent3 = Agent('TestAgent3')
agent4 = Agent('TestAgent4')

# Assign platform for each agent
a2a_platform1 = agent1.use_a2a_platform()
a2a_platform2 = agent2.use_a2a_platform()
a2a_platform3 = agent3.use_a2a_platform()
a2a_platform4 = agent4.use_a2a_platform()

# Provide an ID for each platform (also called as agent id)
registry.register('EchoAgent', a2a_platform1)
registry.register('SummationAgent', a2a_platform2)
registry.register('OrchAgent', a2a_platform3)
registry.register('FinalSumAgent', a2a_platform4)

# Following prints show how to get basic agent related details.
# print(f"Total registered agents: {registry.count()}")
# print(registry.get("EchoAgent")._agent.name)

```

User defined methods. Delays are added to mimic LLMs response time and to watch different status - PENDING, RUNNING, DONE and ERROR. Delay time can be increased if you want to do this slowly and observe what is happening.

```
async def echo(msg: str):
```

```
    '''
```

```
    A simple echo method that waits for 30 seconds before returning the input message.
```

```
    '''
```

```
    if not isinstance(msg, str):
```

```
        raise ValueError("msg must be a string")
```

```
    await asyncio.sleep(30)
```

```
    return f"message is: {msg}"
```

```
async def do_summation(num1: int, num2: int):
```

```
    '''
```

```
    A simple summation method that waits for 30 seconds before returning the sum of two numbers.
```

```
    '''
```

```
    if not isinstance(num1, int) or not isinstance(num2, int):
```

```
        raise ValueError("Please enter integers only")
```

```
    await asyncio.sleep(30)
```

```
    return f"{num1+num2}"
```

```
async def final_summation(mysum: int, num1: int):
```

```
    '''
```

```
    A simple summation method that waits for 20 seconds before returning the sum of two numbers.
```

```
    '''
```

```
    if not isinstance(mysum, int) or not isinstance(num1, int):
```

```
        raise ValueError("numbers must be an integer")
```

```
    await asyncio.sleep(20)
```

```
    return f"{mysum+num1}"
```

#-----

```
async def await_subtask_result(orchestration_task, subtask,
```

```
poll_interval=0.1):
```

```
    '''
```

```
    This is an internal and private helper function to await a subtask's result within an orchestration task.
```

```
    '''
```

```
    while True:
```

```
        for st in orchestration_task.result.get("subtasks", []):
```

```
            if st["task_id"] == subtask["task_id"]:
```

```
                if st["status"] in ["DONE", "ERROR"]:
```

```
                    return st.get("result")
```

```
                break
```

```
        await asyncio.sleep(poll_interval)
```

```

# Give an ID for each user-defined method and register the methods on
whichever platform that needs to access those methods.
a2a_platform1.router.register("echo_message", echo)
a2a_platform2.router.register("do_summation", do_summation)
a2a_platform4.router.register("do_summation", do_summation)
a2a_platform4.router.register("final_summation", final_summation)

# add capabilities, descriptions and examples for each platform
a2a_platform1.add_capabilities('Prints the entered message')
a2a_platform1.add_descriptions(['Waits for 30 seconds and then provides the
entered message'])

a2a_platform1.populate_methods_from_router()
a2a_platform1.add_examples(['To execute "echo_message" method': 'curl -X
POST http://localhost:8000/a2a -H "Content-Type: application/json" -d
{"jsonrpc": "2.0", "agent_id": "EchoAgent",
"method": "create_task_and_run", "params": {"method": "echo_message",
"params": {"msg": "Hello"}}, "id": 1}'], 'To get status of the task
with task_id': 'curl -X POST http://localhost:8000/a2a -H "Content-Type:
application/json" -d
{"jsonrpc": "2.0", "agent_id": "EchoAgent", "method": "task_status",
"params": {"task_id": "<task_id>"}, "id": 2}'], 'To view the status of
tasks in a browser': 'http://localhost:8000/agents/EchoAgent/tasks'])

a2a_platform2.add_capabilities('Prints summation of two numbers')
a2a_platform2.add_descriptions(['Waits for 30 seconds and then provides the
summation of two entered numbers'])
a2a_platform2.populate_methods_from_router()
a2a_platform2.add_examples(['To execute "do_summation" method': 'curl -X
POST http://localhost:8000/a2a -H "Content-Type: application/json" -d
{"jsonrpc": "2.0", "agent_id": "SummationAgent",
"method": "create_task_and_run", "params": {"method": "do_summation",
"params": {"int1": 2, "int2": 4}}, "id": 1}'], 'To get status of the
task with task_id using curl': 'curl -X POST http://localhost:8000/a2a -H
"Content-Type: application/json" -d
{"jsonrpc": "2.0", "agent_id": "SummationAgent", "method": "task_sta
tus", "params": {"task_id": "<task_id>"}, "id": 2}'], 'To view the
status of tasks in a browser':
'http://localhost:8000/agents/SummationAgent/tasks'])

a2a_platform4.add_capabilities('Displays the summation result')
a2a_platform4.add_descriptions(['Gets two numbers, waits for 20 seconds,
adds two numbers, and prints the summation'])
a2a_platform4.add_methods([{"name": "do_summation", "description": "Waits
for 30 seconds and provides the summation of two numbers provided as
input."},
{"name": "final_summation", "description":
"Waits for 20 seconds and provides the summation of two numbers provided as
input."}])
a2a_platform4.add_examples(['To get results from SummationAgent (with
do_summation) and add it to another number within "final_summation"
method': 'curl -X POST http://localhost:8000/a2a -H "Content-Type:
application/json" -d

```

```
"{"jsonrpc":"2.0","id":1,"agent_id":"OrchAgent","method":"orchestrate_tasks_tracked_seq","params":{"msg":"Hello from orchestration","num1":3,"num2":12,"num3":12}}", 'To get status of the task with task_id': 'curl -X POST http://localhost:8000/a2a -H "Content-Type: application/json" -d {"jsonrpc":"2.0","agent_id":"OrchAgent","method":"task_status","params":{"task_id":"<task_id>"},"id":2}}', 'To view the status of tasks in a browser': 'http://localhost:8000/agents/OrchAgent/tasks']])
```

Example 1: Independent

'''Just pass the curl command for a single agent in the terminal and follow its task status using curl command or in a browser as given in example of a2a_platform1 or a2a_platform2. Multiple agents can be executed in parallel (at the same time) as everything is asynchronous.

Give task: curl -X POST http://localhost:8000/a2a -H "Content-Type: application/json" -d '{"jsonrpc":"2.0","agent_id":"EchoAgent","method":"create_task_and_run","params":{"method":"echo_message","params":{"msg":"Hello"},"id":1}}'

Get status (replace task_id): curl -X POST http://localhost:8000/a2a -H "Content-Type: application/json" -d '{"jsonrpc":"2.0","agent_id":"SummationAgent","method":"task_status","params":{"task_id":"<task_id>"},"id":2}'

Browser: http://localhost:8000/agents/EchoAgent/tasks

'''

#-----

Example 2: De-centralised

Agent A invoking Agent B (A => B)

For agent-agent orchestration (agent calling another agent), register the orchestration methods in each agent's platform router.

This enables an agent (e.g., EchoAgent) to call another agent (e.g., SummationAgent).

```
for agent_id, platform in registry._agents.items():
    if hasattr(platform, "router"):
        platform.router.register_orchestration_methods(platform, registry)
```

'''

Give task: curl -X POST http://localhost:8000/a2a -H "Content-Type: application/json" -d '{"jsonrpc":"2.0","id":1,"agent_id":"EchoAgent","method":"call_agent","params":{"target_agent_id":"SummationAgent","method":"do_summation","params":{"num1":3,"num2":4}}}'

Get status: curl -X POST http://localhost:8000/a2a -H "Content-Type: application/json" -d

```
"{"jsonrpc":"2.0","agent_id":"SummationAgent","method":"task_status","params":{"task_id":"<task_id>"},"id":2}"
```

Browser: <http://localhost:8000/agents/SummationAgent/tasks>

'''

```
#-----  
-----
```

Example 3: Parallel without task_id for OrchAgent

Agent A and B are executed in parallel (A || B) by a third agent

Separate agent for orchestration (only orchestration, no task registration). Task status can be monitored in respective agent's endpoint

async def orchestrate_echo_and_sum(platform, params, registry):

'''

Orchestrates EchoAgent and SummationAgent tasks.

params: dict containing {'msg': str, 'num1': int, 'num2': int}

'''

echo_task = await platform.rpc_call_agent(

"EchoAgent",

"echo_message",

{"msg": params["msg"]},

registry

)

sum_task = await platform.rpc_call_agent(

"SummationAgent",

"do_summation",

{"num1": params["num1"], "num2": params["num2"]},

registry

)

return {"echo_task": echo_task, "sum_task": sum_task}

a2a_platform3.register_orchestration_task_on_resp_agent("orchestrate_tasks",
orchestrate_echo_and_sum, registry)

'''

Give task: curl -X POST <http://localhost:8000/a2a> -H "Content-Type:

application/json" -d

```
"{"jsonrpc":"2.0","id":1,"agent_id":"OrchAgent","method":"orchestrate_tasks","params":{"msg":"Hello","num1":3,"num2":5}}"
```

Get status: For echo task: curl -X POST <http://localhost:8000/a2a> -H

"Content-Type: application/json" -d

```
"{"jsonrpc":"2.0","agent_id":"EchoAgent","method":"task_status","params":{"task_id":"<task_id>"},"id":2}"
```

For sum task: curl -X POST <http://localhost:8000/a2a> -H

"Content-Type: application/json" -d

```
"{"jsonrpc":"2.0","agent_id":"SummationAgent","method":"task_status","params":{"task_id":"<task_id>"},"id":3}"
```

Browser: <http://localhost:8000/agents/EchoAgent/tasks>

<http://localhost:8000/agents/SummationAgent/tasks>

'''

```

#-----
-----

# Example 4: Parallel with task_id for OrchAgent
#*****
# Agent A and B are executed in parallel (A || B)
# Separate agent for orchestration (also has its own registered tasks)
async def orchestrate_echo_and_sum_tracked(platform, params, registry,
tracked_call, orchestration_task):
    '''
    Orchestrates EchoAgent and SummationAgent tasks.
    params: dict containing {'msg': str, 'num1': int, 'num2': int}
    '''
    await tracked_call(
        "EchoAgent",
        "echo_message",
        {"msg": params["msg"]},
        registry
    )
    await tracked_call(
        "SummationAgent",
        "do_summation",
        {"num1": params["num1"], "num2": params["num2"]},
        registry
    )

    # Enable the following lines if the following behaviour is wanted:
    # Under the orchestration result, displays each agent's results (mostly
duplicate of what is found in subtasks).
    # orchestration_result = {}
    # for st in orchestration_task.result.get("subtasks", []):
    #     if st["agent_id"] == "EchoAgent":
    #         orchestration_result["echo_task"] = st
    #     elif st["agent_id"] == "SummationAgent":
    #         orchestration_result["sum_task"] = st
    # orchestration_result["pipeline"] = "Echo and Summation in parallel."

    # return orchestration_result
    return {}

a2a_platform3.register_orchestration_as_task("orchestrate_tasks_tracked",
orchestrate_echo_and_sum_tracked, registry)

'''
Give task: curl -X POST http://localhost:8000/a2a -H "Content-Type:
application/json" -d
{"\"jsonrpc\": \"2.0\", \"id\": 1, \"agent_id\": \"OrchAgent\", \"method\": \"orch
estrate_tasks_tracked\", \"params\": {\"msg\": \"Hello\", \"num1\": 3, \"num2\": 5
}}"

Get status: curl -X POST http://localhost:8000/a2a -H "Content-Type:
application/json" -d

```

```
"{\\"jsonrpc\\":\\"2.0\\",\\"agent_id\\":\\"OrchAgent\\",\\"method\\":\\"task_status\\"  
,\\"params\\":{\\"task_id\\":\\"<task_id>\\",\\"id\\":2}}
```

Browser: http://localhost:8000/agents/OrchAgent/tasks

'''

```
#-----  
-----
```

Example 5: Hybrid

Agent A || B -> C

Separate agent for orchestration (also has its own registered tasks)

async def orchestrate_echo_sum_display_seq_tracked(platform, params,

registry, tracked_call, orchestration_task):

'''

Orchestrates EchoAgent and SummationAgent tasks.

params: dict containing {'msg': str, 'num1': int, 'num2': int}

'''

await tracked_call(

"EchoAgent",

"echo_message",

{"msg": params["msg"]},

registry

)

sum_task = await tracked_call(

"SummationAgent",

"do_summation",

{"num1": params["num1"], "num2": params["num2"]},

registry

)

sum_result = await await_subtask_result(orchestration_task, sum_task,

poll_interval=0.2)

await tracked_call(

"FinalSumAgent",

"final_summation",

{"mysum": int(sum_result), "num1": params["num3"]},

registry

)

orchestration_result = {}

for st in orchestration_task.result.get("subtasks", []):

if st["agent_id"] == "EchoAgent":

orchestration_result["echo_task"] = st

elif st["agent_id"] == "SummationAgent":

orchestration_result["sum_task"] = st

orchestration_result["pipeline"] = "Echo and Summation in parallel."

return orchestration_result

return {}

a2a_platform3.register_orchestration_as_task("orchestrate_tasks_tracked_seq
", orchestrate_echo_sum_display_seq_tracked, registry)

'''

```
Give task: curl -X POST http://localhost:8000/a2a -H "Content-Type:
application/json" -d
{"jsonrpc":"2.0","id":1,"agent_id":"OrchAgent","method":"orch
estrate_tasks_tracked_seq","params":{"msg":"Hello from
orchestration","num1":3,"num2":12,"num3":10}}
```

```
Get status: curl -X POST http://localhost:8000/a2a -H "Content-Type:
application/json" -d
{"jsonrpc":"2.0","agent_id":"OrchAgent","method":"task_status"
,"params":{"task_id":"<task_id>"},"id":2}
```

Browser: <http://localhost:8000/agents/OrchAgent/tasks>

'''

#-----

```
# Run the platform with registry containing registered agents.
# app = create_app(registry=registry)
# web.run_app(app, port=8000)
```

```
async def _shutdown(app: web.Application):
    for task in asyncio.all_tasks():
        if task is not asyncio.current_task():
            task.cancel()
    await app.shutdown()
    await app.cleanup()
```

```
async def _main():
    app = create_app(registry=registry)
    runner = web.AppRunner(app)
    await runner.setup()
    site = web.TCPSite(runner, "0.0.0.0", 8000)
    await site.start()

    try:
        await asyncio.Event().wait()
    except (KeyboardInterrupt, asyncio.CancelledError):
        logger.info("Server shutdown requested (Ctrl+C pressed).")
        await _shutdown(app)
    finally:
        await runner.cleanup()
```

```
if __name__ == "__main__":
    logger.info("Press (Ctrl+C) to stop the server.")
    asyncio.run(_main())
```